/* 2) Write a C program that contains a string (char pointer) with a value "Helloworld". The program should AND or XOR each character in this string with 127 and display the result. */

/* ===== 1) XOR ===== */

```
#include <stdio.h>
#include <string.h>

int main()
{
   char String[] = "Helloworld";
char cipher[100];     char
plain[100];     int len =
strlen(String);

   for (int i = 0; i < len; i++)
   {
      cipher[i] = String[i] ^ 127;
   }     cipher[len] = '\0';     printf("Cipher
text: %s\n", cipher);

   for (int i = 0; i < len; i++)
   {        plain[i] = cipher[i] ^
127;
   }     plain[len] = '\0';     printf("Plain
text: %s\n", plain);

   return 0;
}
```

/* Output:-
Cipher text: 7<non-printable characters>
Plain text: Helloworld
*/

/* ===== 2) AND ===== */

```
#include <stdio.h>
#include <string.h>

int main()
{
```

```
   char String[] = "Helloworld";
   char cipher[100];     char
plain[100];     int len =
strlen(String);

   for (int i = 0; i < len; i++)
   {
      cipher[i] = String[i] & 127;
   }     cipher[len] = '\0';     printf("Cipher
text: %s\n", cipher);

   for (int i = 0; i < len; i++)
   {
      plain[i] = cipher[i] & 127;
   }     plain[len] = '\0';     printf("Plain
text: %s\n", plain);

   return 0;
}

/* Output:-
Cipher text: Helloworld
Plain text: Helloworld
*/
```

9. Write a Java program to perform encryption and decryption using Ceaser Cipher.

```
import java.util.*;

class Solution {
   static String cipher = "";
   static String plain_text = "";

   public String Encryption(String pt, int key) {
      for (char c : pt.toCharArray()) {
         if (Character.isUpperCase(c)) {
            cipher += (char) ((c - 'A' + key) % 26 + 'A');
         } else {
            cipher += c;
         }
      }
      return cipher;
   }

   public String Decryption(String en, int key) {
      for (char c : en.toCharArray()) {
         if (Character.isUpperCase(c)) {
            plain_text += (char) ((c - 'A' - key + 26) % 26 + 'A');
         } else if (Character.isLowerCase(c)) {
            plain_text += c;
         }
      }
      return plain_text;
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.println("Enter plaintext:");
      String pt = sc.nextLine();

      System.out.println("Enter Key:");
      int key = sc.nextInt();

      String encryptedText = Encryption(pt, key);
      System.out.println("Encrypted text: " + encryptedText);

      String decryptedText = Decryption(encryptedText, key);
```

```
        System.out.println("Decrypted  Text: " + decryptedText);

        sc.close();
    }
}
```

Output

Enter plaintext:

Kits college

Enter Key:

3

Encrypted text: Nlwv college

Decrypted Text: Kits college

12. Implement the Diffie-Hellman Key Exchange mechanism using

```
import java.math.BigInteger;
import java.util.*;
public class DH {
public static void main(String args[]){
Scanner sc=new Scanner(System.in);
System.out.println("Enter prime no q");
BigInteger q=sc.nextBigInteger();
System.out.println("Enter prime no a");
BigInteger a=sc.nextBigInteger();
System.out.println("Enter private key xa");
BigInteger xa=sc.nextBigInteger();
System.out.println("Enter private key xb");
BigInteger xb=sc.nextBigInteger();

BigInteger ya=a.modPow(xa,q);
BigInteger yb=a.modPow(xb,q);
BigInteger Ka=yb.modPow(xa,q);
BigInteger Kb=ya.modPow(xb,q);
System.out.println("Prime no q="+q);
System.out.println("Primitive root of pis a="+a);
System.out.println("user A private key xa="+xa);
System.out.println("user B private key xb="+xb);
System.out.println("user A public key ya="+ya);
System.out.println("user B public key yb="+yb);
System.out.println("user A Secret key Ka="+Ka);
System.out.println("user A Secret key Kb="+Kb);
}
}
```

Output

```
Enter prime no q
23
Enter prime no a
5
Enter private key xa
6
Enter private key xb
15

Prime no q=23
Primitive root of pis a=5
user A private key xa=6
user B private key xb=15
user A public key ya=8
user B public key yb=19
user A Secret key Ka=2
user A Secret key Kb=2
```